

3.10 Homework for Spring Data Jpa

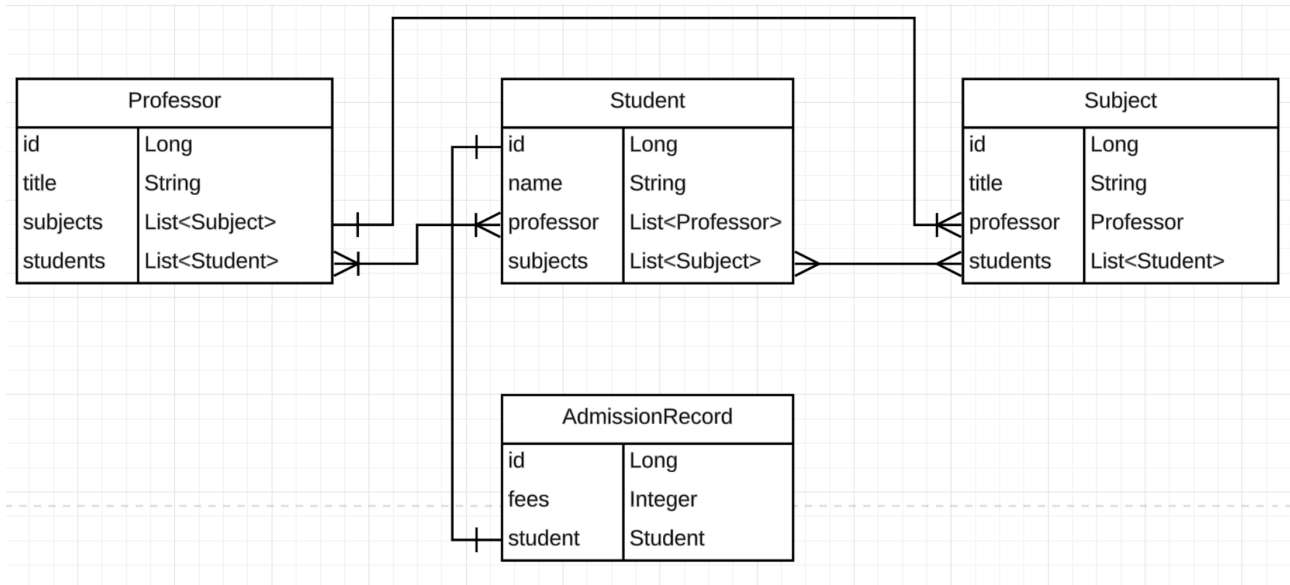
- List of all the annotations in the Spring Data JPA:

Category	Annotation	Function / Purpose
Entity Mapping	@Entity	Marks class as database table
	@Id	Marks primary key
	@GeneratedValue	Auto-generates primary key
	@Table	Customizes table name
	@Column	Customizes column properties
Relationship Mapping	@OneToOne	One-to-one relationship
	@OneToMany	One-to-many relationship
	@ManyToOne	Many-to-one relationship
	@ManyToMany	Many-to-many relationship
	@JoinColumn	Defines foreign key column
	@JoinTable	Defines join table (ManyToMany)
	mappedBy	Marks inverse side of relationship
	FetchType.LAZY	Loads related data on demand
	FetchType.EAGER	Loads related data immediately
	cascade	Propagates operations to children
Repository Layer	orphanRemoval	Deletes orphan child entity
	@Repository	Marks DAO , enables exception translation
	@Query	Defines custom JPQL/SQL query
	@EntityGraph	Controls fetch strategy
Transaction Management	@Transactional	Manages database transactions

3.10 Homework for Spring Data Jpa

- **Build this College management system in Spring Data JPA with all the mappings as well as APIs to assign the various mappings:**

- **ER diagram :**



- **Link:** <https://github.com/sibasundarj8/CollegeManagementSystem>

- **Project Overview:**

- The **College Management System** is a backend REST API developed using **Spring Boot** and **Spring Data JPA** to manage relationships between:
 - Students
 - Subjects
 - Professors
 - Admission Records
- The system demonstrates advanced **ORM mapping**, **RESTful API design**, **DTO-based architecture**, and **query optimization techniques**.
- The goal of the project is to implement **efficient relational data management** while avoiding common ORM pitfalls such as **N+1 query problems**, **lazy loading issues**, and **unnecessary entity traversal**.

- **Technology Stack:**

Technology	Purpose
Spring Boot	Backend framework
Spring Data JPA	Data access layer
Hibernate	ORM implementation
MySQL	Relational database
Jackson	JSON serialization
Jakarta Validation	Request validation
Lombok	Boilerplate reduction

3.10 Homework for Spring Data Jpa

• DTO Based Architecture:

The project uses **DTOs** instead of exposing **entities** to the **API**.
This avoids:

- **Lazy loading** issues
- **Circular** JSON serialization
- **Over-fetching** database data

• Request DTOs

Used to **receive API input**.

Example:

```
public class StudentRequestDto {  
    private String name;  
    private String registrationNumber;  
    private List<Long> subjectIds;  
    private Integer fees;  
}
```

• Response DTOs

Used to **send structured API responses**.

Example:

```
public record StudentResponseDto(  
    Long id,  
    String name,  
    String registrationNumber,  
    Set<SubjectResponseDto> subjects  
) {}
```

• Flat DTOs

Used to **optimize complex joins**.

Example:

```
public record StudentFlatDto(  
    Long id,  
    String name,  
    String registrationNumber,  
    String subject,  
    String professor  
) {}
```

This technique prevents **N+1 queries** during read operations.

• REST API Design:

• Subject APIs

Method	Endpoint	Description
GET	/subjects	Get all subjects
GET	/subjects/{id}	Get subject by ID
GET	/subjects?title={title}	Search subject by title
GET	/subjects/{id}/students	Get students enrolled in a subject
POST	/subjects	Create a new subject
PUT	/subjects/{subjectId}/professor/{professorId}	Assign professor to subject
DELETE	/subjects/{subjectId}/professor	Unassign professor from subject
DELETE	/subjects/{id}	Delete a subject

3.10 Homework for Spring Data Jpa

• Student APIs

Method	Endpoint	Description
GET	/students	Get all students
GET	/students?name={name}	Search students by name
GET	/students/{id}	Get student by ID
GET	/students/registration/{registrationNumber}	Get student by registration number
GET	/students/{id}/subjects	Get all subjects of a student
POST	/students	Create a new student
POST	/students/{studentId}/subjects/{subjectId}	Assign subject to student
DELETE	/students/{studentId}/subjects/{subjectId}	Unassign subject from student

• Student APIs

Method	Endpoint	Description
GET	/professors	Get all professors
GET	/professors/{id}	Get professor by ID
GET	/professors?title={title}	Search professor by title
GET	/professors/{id}/subjects	Get subjects taught by professor
POST	/professors	Create a new professor

• What Was Done Exceptionally Well:

This project demonstrates several **advanced backend practices**.

• Excellent Use of DTO Layer

Entities are never exposed directly to the API.

Benefits:

- Prevents **infinite JSON recursion**
- **Improves security**
- Allows flexible response design

• Understanding of Hibernate Internals

The implementation correctly handles:

- Lazy loading
- Persistence context
- Dirty checking
- Bidirectional relationship synchronization

• Avoidance of N+1 Query Problems

- By using **flat DTO projections**, complex data can be fetched efficiently.
- This significantly improves performance.

• Proper REST API Design

Correct HTTP semantics are followed:

- **POST** → create resource
- **PUT** → update relationship
- **DELETE** → remove association

3.10 Homework for Spring Data Jpa

- **Clean Transactional Boundaries**

- Business logic is correctly placed inside **service layer transactions**.

- **Database Index Optimization**

Indexes are defined for:

- student name
- subject code
- join table foreign keys

This improves query performance.

- **Conclusion:**

- This project successfully demonstrates a well-structured **Spring Boot REST backend** with proper usage of:
 - JPA entity relationships
 - DTO architecture
 - RESTful API design
 - Hibernate performance optimization techniques
- It highlights practical solutions to common ORM challenges such as **lazy loading, N+1 queries, and entity graph traversal**, making it a strong example of backend engineering practices.

- **Link:** <https://github.com/sibasundarj8/CollegeManagementSystem>